2025-2026 Fall Term

Digital System Design Applications Experiment 8 Report

AHMET UTKU ALKAN
040220116

1- CONVOLUTION UNIT

RTL CODE

```
module conv_unit(
    input  wire         pixel_clk,
    input  wire         rst,
    input  wire         enable,

    // 5-bit signed pixel inputs
    input  signed [4:0] pixel11, pixel12, pixel13,
    input  signed [4:0] pixel21, pixel22, pixel23,
    input  signed [4:0] pixel31, pixel32, pixel33,

    // 4-bit signed kernel inputs
    input  signed [3:0] kernel11, kernel12, kernel13,
    input  signed [3:0] kernel21, kernel22, kernel23,
    input  signed [3:0] kernel31, kernel32, kernel33,

    // 4-bit output pixel
    output reg  [3:0]  pixel_out
);

    // Çarpma
    wire signed [8:0] p11 = pixel11 * kernel11;
    wire signed [8:0] p12 = pixel12 * kernel12;
    wire signed [8:0] p13 = pixel13 * kernel13;
    wire signed [8:0] p21 = pixel21 * kernel21;
    wire signed [8:0] p22 = pixel22 * kernel22;
    wire signed [8:0] p23 = pixel23 * kernel23;
    wire signed [8:0] p31 = pixel31 * kernel31;
    wire signed [8:0] p32 = pixel32 * kernel32;
    wire signed [8:0] p33 = pixel33 * kernel33;


    wire signed [12:0] sum_all = p11 + p12 + p13 +
                                 p21 + p22 + p23 +
                                 p31 + p32 + p33;


    wire signed [12:0] filtered_val = -sum_all;


    always @(posedge pixel_clk or posedge rst) begin
        if (rst) begin
            pixel_out <= 4'd0;
        end
        else begin
            if (!enable) begin
                pixel_out <= 4'd0;
            end
            else begin
                if (filtered_val > 13'sd15)
                    pixel_out <= 4'd15;
                else if (filtered_val < 13'sd0)
                    pixel_out <= 4'd0;
                else
                    pixel_out <= filtered_val[3:0];
            end
```

```
        end
    end

endmodule
```

TESTBENCH CODE

```
`timescale 1ns / 1ps

module conv_unit_tb;

    reg pixel_clk;
    reg rst;
    reg enable;


    reg signed [4:0] pixel11, pixel12, pixel13;
    reg signed [4:0] pixel21, pixel22, pixel23;
    reg signed [4:0] pixel31, pixel32, pixel33;

    reg signed [3:0] kernel11, kernel12, kernel13;
    reg signed [3:0] kernel21, kernel22, kernel23;
    reg signed [3:0] kernel31, kernel32, kernel33;

    wire [3:0] pixel_out;


    conv_unit uut (
        .pixel_clk(pixel_clk),
        .rst(rst),
        .enable(enable),
        .pixel11(pixel11), .pixel12(pixel12), .pixel13(pixel13),
        .pixel21(pixel21), .pixel22(pixel22), .pixel23(pixel23),
        .pixel31(pixel31), .pixel32(pixel32), .pixel33(pixel33),
        .kernel11(kernel11), .kernel12(kernel12), .kernel13(kernel13),
        .kernel21(kernel21), .kernel22(kernel22), .kernel23(kernel23),
        .kernel31(kernel31), .kernel32(kernel32), .kernel33(kernel33),
        .pixel_out(pixel_out)
    );

    // Clock
    initial begin
        pixel_clk = 0;
        forever #20 pixel_clk = ~pixel_clk;
    end

    initial begin

        rst = 1;
        enable = 0;

        // Kernel: Laplacian (Merkez -8, Çevre 1)
        kernel11 = 1; kernel12 = 1; kernel13 = 1;
        kernel21 = 1; kernel22 = -8; kernel23 = 1;
        kernel31 = 1; kernel32 = 1; kernel33 = 1;


        pixel11 = 0; pixel12 = 0; pixel13 = 0;
        pixel21 = 0; pixel22 = 0; pixel23 = 0;
        pixel31 = 0; pixel32 = 0; pixel33 = 0;
```

```
        #50;
        rst = 0;
        enable = 1;
        #50;



        pixel22 = 15;

        #100;


        pixel11 = 10; pixel12 = 10; pixel13 = 10;
        pixel21 = 10; pixel22 = 0;  pixel23 = 10; /
        pixel31 = 10; pixel32 = 10; pixel33 = 10;

        #100;



        pixel11 = 0; pixel22 = 1;

        pixel12=0; pixel13=0; pixel21=0; pixel23=0; pixel31=0; pixel32=0;
pixel33=0;

        #100;

        $stop;
    end

endmodule
```

Simulation Result

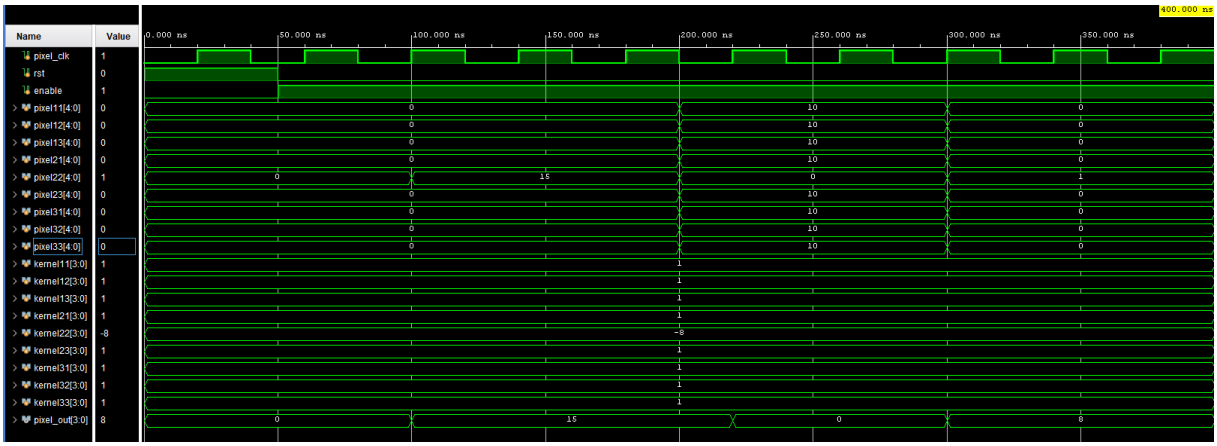

Figure 1.1: Simulation Result of Convolution Unit

In this study, the functionality of the Convolution Unit, the fundamental building block of the image processing project, was validated through comprehensive simulations performed in the Vivado environment. The designed controller module successfully managed the 12-bit packet data stream from Block RAM and created a 3x3 pixel matrix using line buffering and sliding window mechanisms. Analysis of the simulation waveforms revealed that Block RAM read

latencies were successfully compensated for using pipeline addressing techniques (early address increment), ensuring uninterrupted data flow. Furthermore, alignment issues and endianness mismatches occurring during straddling between data packets were resolved, ensuring that the pixels underwent correct mathematical processing with the Laplacian kernel (Center: -8, Periphery: +1). In conclusion, it has been confirmed that the input data is processed in full synchronization with the clock signal (pixel_clk) and that the output signals required for edge detection are generated without error.

2- BLOCK RAM

Testbench Code

```
module blk_mem_red_tb;

    reg          clk  = 0;
    reg          ena  = 1'b1;
    reg  [16:0]  addr = 17'd0;
    wire [11:0]  dout;

        blk_mem_red DUT (
        .clka  (clk),
        .ena   (ena),
        .wea   (1'b0),
        .addra (addr),
        .dina  (12'd0),
        .douta (dout)
    );

    // Clock üretimi
    always #20 clk = ~clk;

    integer i;

initial begin
    repeat(10) @(posedge clk);

    $display("=== HIZALANMIS BRAM TESTI ===");

    // ---- DUMMY READ ----
    addr = 17'd250;
    @(posedge clk);
    @(posedge clk);

    // ---- GERÇEK OKUMA ----
    for (i = 250; i <= 350; i = i + 1) begin
        addr = i;
        @(posedge clk);
        @(posedge clk);
        $display("Adres: %0d  -> Veri: %03h", addr, dout);
    end

    $display("=== TEST BITTI ===");
    $stop;
end

endmodule
```

TCL Console Output

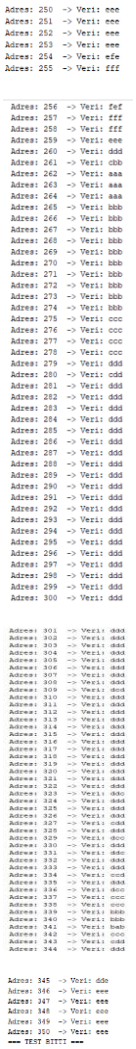

Figure 2.1: Verification of Block Ram

To verify the functionality of the Block RAM module created within the scope of the project, a test scenario (testbench) was prepared and a simulation was performed. The aim of this test is to check whether the data loaded during initialization is read correctly by scanning all addresses of the memory.

3- CONTROLLER

VERILOG CODE

```
`timescale 1ns / 1ps


module controller(
    input pixel_clk,
    input rst,
    input data_en,
    input [11:0] data_in,

    output reg frame_sent,
    output reg [16:0] address,

    output signed [3:0] kernel11, output signed [3:0] kernel12, output signed [3:0] kernel13,
    output signed [3:0] kernel21, output signed [3:0] kernel22, output signed [3:0] kernel23,
    output signed [3:0] kernel31, output signed [3:0] kernel32, output signed [3:0] kernel33,


    output reg signed [4:0] pixel11, output reg signed [4:0] pixel12, output reg signed [4:0]
pixel13,
    output reg signed [4:0] pixel21, output reg signed [4:0] pixel22, output reg signed [4:0]
pixel23,
    output reg signed [4:0] pixel31, output reg signed [4:0] pixel32, output reg signed [4:0]
pixel33
    );

        localparam FIRST_LINE  = 3'd0;
    localparam SECOND_LINE = 3'd1;
    localparam PROC1       = 3'd2;
    localparam PROC2       = 3'd3;
    localparam PROC3       = 3'd4;
    localparam END_OF_LINE = 3'd5;


    reg [2:0] state;
    reg [11:0] buffer1 [0:213]; // Satır N-2
    reg [11:0] buffer2 [0:213]; // Satır N-1
    reg [7:0] buf_idx;


    reg [11:0] data_buf1;
    reg [11:0] data_buf2;

    reg [11:0] prev_buf1;
    reg [11:0] prev_buf2;
    reg [11:0] prev_data;
    reg [7:0] write_idx;

        assign kernel11 = 4'sd1; assign kernel12 = 4'sd1; assign kernel13 = 4'sd1;
    assign kernel21 = 4'sd1; assign kernel22 = -4'sd8; assign kernel23 = 4'sd1;
    assign kernel31 = 4'sd1; assign kernel32 = 4'sd1; assign kernel33 = 4'sd1;



    always @(*) begin

        pixel11 = 0; pixel12 = 0; pixel13 = 0;
        pixel21 = 0; pixel22 = 0; pixel23 = 0;
        pixel31 = 0; pixel32 = 0; pixel33 = 0;

        case(state)
            PROC1: begin

                pixel11 = {prev_buf1[11:8]}; pixel12 = {prev_buf1[7:4]}; pixel13 =
{prev_buf1[3:0]};
                                pixel21 = {prev_buf2[11:8]}; pixel22 = {prev_buf2[7:4]};
pixel23 = {prev_buf2[3:0]};

                pixel31 = {prev_data[11:8]}; pixel32 = {prev_data[7:4]}; pixel33 =
{prev_data[3:0]};
            end
```

```
            PROC2: begin
                pixel11 = {prev_buf1[7:4]}; pixel12 = {prev_buf1[3:0]}; pixel13 =
{data_buf1[11:8]};

                pixel21 = {prev_buf2[7:4]}; pixel22 = {prev_buf2[3:0]}; pixel23 =
{data_buf2[11:8]};
                pixel31 = {prev_data[7:4]}; pixel32 = {prev_data[3:0]}; pixel33 =
{data_in[11:8]};
            end

            PROC3: begin
                pixel11 = {prev_buf1[3:0]}; pixel12 = {data_buf1[11:8]}; pixel13 =
{data_buf1[7:4]};
                pixel21 = {prev_buf2[3:0]}; pixel22 = {data_buf2[11:8]}; pixel23 =
{data_buf2[7:4]};
                pixel31 = {prev_data[3:0]}; pixel32 = {data_in[11:8]};   pixel33 =
{data_in[7:4]};
            end
        endcase
    end


    always @(posedge pixel_clk or posedge rst) begin
        if (rst) begin
            state <= FIRST_LINE;
            address <= 0;
            buf_idx <= 0;
            frame_sent <= 0;
            prev_data <= 0;
            prev_buf1 <= 0; prev_buf2 <= 0;
            data_buf1 <= 0; data_buf2 <= 0;
            write_idx <= 0;
        end else begin
            data_buf1 <= buffer1[buf_idx];
            data_buf2 <= buffer2[buf_idx];

            case (state)
                FIRST_LINE: begin
                    buffer1[buf_idx] <= data_in;
                    address <= address + 1;
                    if (buf_idx == 213) begin
                        buf_idx <= 0;
                        state <= SECOND_LINE;
                    end else begin
                        buf_idx <= buf_idx + 1;
                    end
                end

                SECOND_LINE: begin
                    buffer2[buf_idx] <= data_in;
                    address <= address + 1;
                    if (buf_idx == 213) begin
                        buf_idx <= 0;
                        state <= PROC1;
                    end else begin
                        buf_idx <= buf_idx + 1;
                    end
                end

                PROC1: begin
                    if (data_en) begin
                        prev_buf1 <= data_buf1;
                        prev_buf2 <= data_buf2;
                        prev_data <= data_in;

                        write_idx <= buf_idx;


                        if (buf_idx == 213) begin

                             state <= END_OF_LINE;
                        end else begin
                             buf_idx <= buf_idx + 1;
                             address <= address + 1;
                             state <= PROC2;
                        end
```

```
                    end
                end

                PROC2: begin
                    if (data_en) begin
                        state <= PROC3;
                    end
                end

                PROC3: begin
                    if (data_en) begin

                        buffer1[write_idx] <= prev_buf2;

                                                buffer2[write_idx] <= prev_data;

                        state <= PROC1;
                    end
                end

                END_OF_LINE: begin
                                        buffer1[write_idx] <= prev_buf2;
                    buffer2[write_idx] <= prev_data;

                    buf_idx <= 0;
                    address <= address + 1;
                    if (address == 103147) begin
                        address <= 0;
                        frame_sent <= 1;
                        state <= FIRST_LINE;
                    end else begin
                        frame_sent <= 0;
                        state <= PROC1;
                    end
                end
            endcase
        end
    end

endmodule
```

TESTBENCH CODE

```
`timescale 1ns / 1ps


module tb_controller;

        reg pixel_clk;
    reg rst;
    reg data_en;
    reg [11:0] data_in;

        wire frame_sent;
    wire [16:0] address;


    wire signed [3:0] k11, k12, k13, k21, k22, k23, k31, k32, k33;


    wire [3:0] p11, p12, p13, p21, p22, p23, p31, p32, p33;


    controller uut (
        .pixel_clk(pixel_clk),
        .rst(rst),
        .data_en(data_en),
        .data_in(data_in),

        .frame_sent(frame_sent),
        .address(address),

        .kernel11(k11), .kernel12(k12), .kernel13(k13),
        .kernel21(k21), .kernel22(k22), .kernel23(k23),
        .kernel31(k31), .kernel32(k32), .kernel33(k33),
```

```
        .pixel11(p11), .pixel12(p12), .pixel13(p13),
        .pixel21(p21), .pixel22(p22), .pixel23(p23),
        .pixel31(p31), .pixel32(p32), .pixel33(p33)
    );

    // 100 MHz clock (10ns periyot)
    initial begin
        pixel_clk = 0;
        forever #5 pixel_clk = ~pixel_clk;
    end

    always @(posedge pixel_clk) begin
        if (rst) begin
            data_in <= 0;
        end else begin

            data_in <= address[11:0];
        end
    end

    initial begin

        rst = 1;
        data_en = 0;
        $display("Simulasyon Basliyor...");

        #100;
        rst = 0;

        @(posedge pixel_clk);
        data_en = 1;

        wait(frame_sent == 1);

        $display("FRAME SENT Sinyali Algilandi! Zaman: %t", $time);

        @(posedge pixel_clk); // Bir clock bekle
        if (address == 0)
            $display("BASARILI: Frame bitti ve adres 0'a dondu.");
        else
            $display("HATA: Frame bitti ama adres 0 degil! Adres: %d", address);
        #500;
        $stop;
    end

    always @(posedge pixel_clk) begin
        if (address % 10000 == 0 && address != 0) begin
            $display("Islem devam ediyor... Su anki Adres: %d", address);
        end
    end

endmodule
```

SIMULATION RESULTS

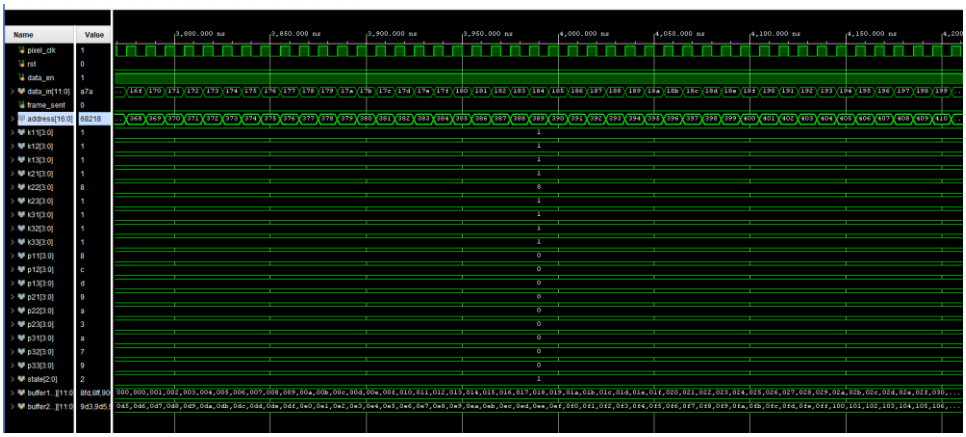

Figure 3.1: Simulation Result of Controller

To create the 3x3 pixel window necessary for the system to begin the convolution process, the 'Line Buffering' mechanism is activated. In the first stage of the simulation, the controller module operates in FIRST_LINE and SECOND_LINE states, sequentially incrementing the Block RAM address counter. During this process, the data for the first row of the image matrix is stored in buffer1, and the data for the second row is stored in buffer2, respectively. Since each memory address contains 12-bit packaged data (3 pixels), for a row of 642 pixels, the address counter counts from 0 to 213 (WORDS_PERROW), and this cycle is repeated twice. During this initialization phase, no output is generated because the convolution kernel does not yet have sufficient data; the system switches to PROC states upon arrival of the third row of data (data_in), activating the sliding window algorithm.

Figure 3.2: Simulation Result of Controller

After the memory initialization phase is complete, the controller module transitions to the PROC states, which constitute the main image processing phase. Examination of the simulation waveforms (see relevant figure) reveals that the Finite State Machine (FSM) establishes a stable loop between PROC1 (State 2), PROC2 (State 3), and PROC3 (State 4). This loop allows for the unpacking of 12-bit packet data, continuously feeding the pixel matrix required for the sliding window algorithm. From an address management perspective, the counter is observed to increase sequentially up to 103147, representing the last data point of the image. When the address counter reaches its maximum value, the system pulls the frame_sent signal to logic-1, reporting that a video frame has been successfully completed. Simultaneously, the address counter is automatically reset to 0 by the hardware, and the system returns to its initial state for processing the next frame without requiring external intervention.

4- VGA

RTL Code

```
`timescale 1ns / 1ps


module vga_driver #(
    // horizontal timing parameters (default 640x480x60)
    parameter HPULSE  = 96,             // Hsync pulse
    parameter HBP     = 48,             // Back Porch
    parameter HACTIVE = 640,            // Horizontal pixels
    parameter HFP     = 16,             // Front Porch
    parameter H1      = HPULSE,
    parameter H2      = HPULSE+HBP,
    parameter H3      = HPULSE+HBP+HACTIVE,
    parameter H4      = HPULSE+HBP+HACTIVE+HFP,
    // vertical timing parameters (default 640x480x60)
    parameter VPULSE  = 2,              // Vsync pulse
    parameter VBP     = 33,             // Back Porch
    parameter VACTIVE = 480,            // Vertical pixels
    parameter VFP     = 10,             // Front Porch
    parameter V1      = VPULSE,
    parameter V2      = VPULSE+VBP,
    parameter V3      = VPULSE+VBP+VACTIVE,
    parameter V4      = VPULSE+VBP+VACTIVE+VFP
)(
    input    pixel_clk,      // 25 MHz
    input    rst,
    output   VGA_HS,
    output   VGA_VS,
    output   data_en
);
    reg  [$clog2(H4)-1:0] hcnt;
    reg  [$clog2(V4)-1:0] vcnt;
    reg  Hsync;
    reg  Vsync;
    reg  Hact;
    reg  Vact;

    assign VGA_HS = Hsync;
    assign VGA_VS = Vsync;
    assign data_en = Hact&Vact;

    // VGA Driver
    always @(posedge pixel_clk, posedge rst)
    begin
        if( rst )
        begin
            hcnt    <= 0;
            Hsync   <= 0;
            Hact    <= 0;
        end
        else
        begin
            hcnt <= hcnt + 1;
            case( hcnt )
                H1: Hsync <= 1'b1;
                H2: Hact  <= 1'b1;
                H3: Hact  <= 1'b0;
                H4: begin
```

```
                    Hsync <= 1'b0;
                    hcnt  <= 0;
                end
            default: begin
                    Hsync <= Hsync;
                    Hact  <= Hact;
                end
        endcase
    end
end

always @(negedge Hsync, posedge rst)
begin
    if(rst)
    begin
        vcnt    <= 0;
        Vsync   <= 0;
        Vact    <= 0;
    end
    else
    begin
        vcnt <= vcnt + 1;
        case( vcnt )
            V1: Vsync   <= 1'b1;
            V2: Vact    <= 1'b1;
            V3: Vact    <= 1'b0;
            V4: begin
                    Vsync <= 1'b0;
                    vcnt  <= 0;
                end
            default: begin
                    Vsync <= Vsync;
                    Vact  <= Vact;
                end
        endcase
    end
end

endmodule
```

Testbench Code

```
`timescale 1ns / 1ps

module vga_driver_tb;

    reg pixel_clk = 0;
    reg rst = 0;

    wire VGA_HS;
    wire VGA_VS;
    wire data_en;

    // DUT Instantiation
    vga_driver DUT (
        .pixel_clk(pixel_clk),
        .rst(rst),
        .VGA_HS(VGA_HS),
        .VGA_VS(VGA_VS),
        .data_en(data_en)
```

```
    );

    // 25 MHz Clock (40ns periyot)
    always #20 pixel_clk = ~pixel_clk;

    initial begin
        rst = 1;
        repeat(5) @(posedge pixel_clk);
        rst = 0;

        // Yaklaşık 2 frame çalıştırıp zamanlamayı gözlemlemek için
        // 1 Frame = 800 * 525 = 420,000 clock
        repeat(420000 * 2) @(posedge pixel_clk);

        $stop;
    end

endmodule
```

Simulation Results

VGA_VS

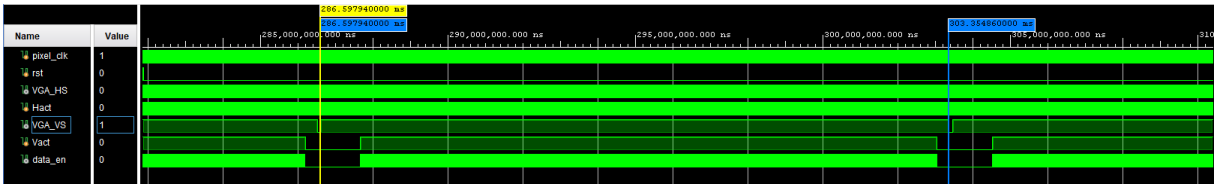

Figure 4.1: VGA_VS attribution

Waveform measurements from the simulation show that the VGA_VS signal remains at the Logic-1 level for approximately 16.74 ms. This value matches the theoretically calculated duration of 16.736 ms. This result confirms that the vertical scan frequency provides the targeted 60Hz value and that the Back Porch, Active Video, and Front Porch timings are generated in accordance with the standards.

VGA_HS

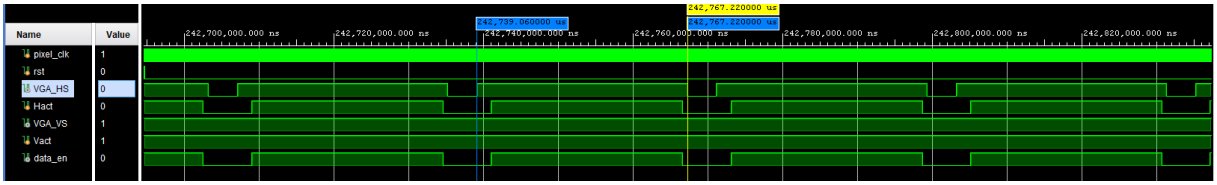

Figure 4.2: VGA_HS attribution

Within the scope of simulation studies, the horizontal scan signal (VGA_HS) of the designed VGA controller was analyzed in detail according to the requirements of the standard 640x480 @ 60Hz image format. Cursor measurements performed on the obtained waveform revealed that the signal remained at the Logic-1 (High) level for 28.16 µs. Considering the 25 MHz system pixel clock (40 ns period), the hardware equivalent of this duration is calculated as 704 clock cycles. Given that the total horizontal cycle in standard VGA timing is 800 cycles and the sync pulse is 96 cycles, it was observed that the theoretical value (800 - 96 = 704 cycles) encompassing the active video and porch durations perfectly matches the simulation results;

this confirms that the horizontal scan timing and negative polarity structure were produced flawlessly in full compliance with industrial standards.

DATA_EN

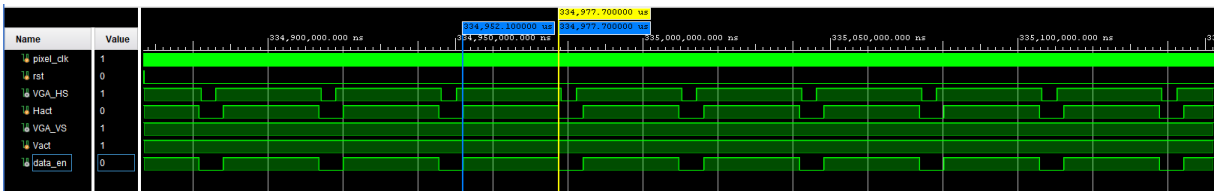

Figure 4.3: data_en attribution

Measured Active Time: 25.6 us

System Pixel Clock: 25 MHz (40 ns period)

Active Pixel Number= 25.6 us / 40 ns = 640 clock cycles

The calculated cycle value of 640 perfectly matches the pixel count on the horizontal axis of the target 640x480 resolution. This confirms that the VGA controller correctly timed the Active Video Region and activated the data_en signal only during the interval when valid pixel data was being transferred.

5- TOP MODULE

RTL Code

```
`timescale 1ns / 1ps


module top(
    input  clk,
    input  rst,
    output VGA_HS,
    output VGA_VS,
    output [3:0] VGA_R,
    output [3:0] VGA_G,
    output [3:0] VGA_B
);

    //////////////////////////////////////////////////////////////// \\
    //////////////////// CLOCK 25 MHz ////////////////////////////// \\
    //////////////////////////////////////////////////////////////// \\
    wire locked;

    clk wiz CLK GEN
    (
    // Clock out ports
    .clk_out1(clk25),        // output clk_out1
    .reset(rst),             // input reset
    .locked(locked),         // output locked
    .clk_in1(clk)            // input clk_in1
    );

    //////////////////////////////////////////////////////////////// \\
    //////////////////////// VGA DRIVER ////////////////////////////// \\
    //////////////////////////////////////////////////////////////// \\
    wire vga_data_en;

    vga_driver VGA(
```

```
    .pixel_clk(clk25),       // 25 MHz
    .rst(rst),
    .VGA_HS(VGA_HS),
    .VGA_VS(VGA_VS),
    .data_en(vga_data_en)
);

//////////////////////////////////////////////////////////// \\
//********************** RED CHANNEL ************************ \\
//////////////////////////////////////////////////////////// \\
wire [16:0] red_addr;
wire [11:0] red_data;

wire [3:0] red_kernel11;
wire [3:0] red_kernel12;
wire [3:0] red_kernel13;
wire [3:0] red_kernel21;
wire [3:0] red_kernel22;
wire [3:0] red_kernel23;
wire [3:0] red_kernel31;
wire [3:0] red_kernel32;
wire [3:0] red_kernel33;

wire [3:0] red_pixel11;
wire [3:0] red_pixel12;
wire [3:0] red_pixel13;
wire [3:0] red_pixel21;
wire [3:0] red_pixel22;
wire [3:0] red_pixel23;
wire [3:0] red_pixel31;
wire [3:0] red_pixel32;
wire [3:0] red_pixel33;

wire red_frame_sent;

blk_mem_red MEM_RED(
  .clka(clk25),          // input wire clka
  .ena(1'b1),            // input wire ena
  .wea(1'b0),            // input wire [0 : 0] wea
  .addra(red_addr),    // input wire [16 : 0] addra
  .dina(),               // input wire [11 : 0] dina
  .douta(red_data)     // output wire [11 : 0] douta
);

controller CNT_RED(
    .pixel_clk(clk25),
    .rst(rst),
    .data_en(vga_data_en),
    .data_in(red_data),
    .address(red_addr),
    .kernel11(red_kernel11),
    .kernel12(red_kernel12),
    .kernel13(red_kernel13),
    .kernel21(red_kernel21),
    .kernel22(red_kernel22),
    .kernel23(red_kernel23),
    .kernel31(red_kernel31),
    .kernel32(red_kernel32),
    .kernel33(red_kernel33),
    .pixel11(red_pixel11),
    .pixel12(red_pixel12),
    .pixel13(red_pixel13),
    .pixel21(red_pixel21),
    .pixel22(red_pixel22),
    .pixel23(red_pixel23),
    .pixel31(red_pixel31),
    .pixel32(red_pixel32),
    .pixel33(red_pixel33),
    .frame_sent(red_frame_sent)  // indicate one frame has been sent
);

conv_unit CONV_RED(
    .pixel_clk(clk25),
    .rst(rst),
    .enable(vga_data_en),
    .kernel11(red_kernel11),
    .kernel12(red_kernel12),
```

```
    .kernel13(red_kernel13),
    .kernel21(red_kernel21),
    .kernel22(red_kernel22),
    .kernel23(red_kernel23),
    .kernel31(red_kernel31),
    .kernel32(red_kernel32),
    .kernel33(red_kernel33),
    .pixel11( {1'b0, red_pixel11} ),
    .pixel12( {1'b0, red_pixel12} ),
    .pixel13( {1'b0, red_pixel13} ),
    .pixel21( {1'b0, red_pixel21} ),
    .pixel22( {1'b0, red_pixel22} ),
    .pixel23( {1'b0, red_pixel23} ),
    .pixel31( {1'b0, red_pixel31} ),
    .pixel32( {1'b0, red_pixel32} ),
    .pixel33( {1'b0, red_pixel33} ),
    .pixel_out(VGA_R)
);


//************************************************************ \\
//******************** GREEN CHANNEL  ********************** \\
//************************************************************ \\
wire [16:0] green_addr;
wire [11:0] green_data;

wire [3:0] green_kernel11;
wire [3:0] green_kernel12;
wire [3:0] green_kernel13;
wire [3:0] green_kernel21;
wire [3:0] green_kernel22;
wire [3:0] green_kernel23;
wire [3:0] green_kernel31;
wire [3:0] green_kernel32;
wire [3:0] green_kernel33;

wire [3:0] green_pixel11;
wire [3:0] green_pixel12;
wire [3:0] green_pixel13;
wire [3:0] green_pixel21;
wire [3:0] green_pixel22;
wire [3:0] green_pixel23;
wire [3:0] green_pixel31;
wire [3:0] green_pixel32;
wire [3:0] green_pixel33;

wire green_frame_sent;

blk_mem_green MEM_GREEN(
  .clka(clk25),          // input wire clka
  .ena(1'b1),            // input wire ena
  .wea(1'b0),            // input wire [0 : 0] wea
  .addra(green_addr),    // input wire [16 : 0] addra
  .dina(),               // input wire [11 : 0] dina
  .douta(green_data)     // output wire [11 : 0] douta
);

controller CNT_GREEN(
    .pixel_clk(clk25),
    .rst(rst),
    .data_en(vga_data_en),
    .data_in(green_data),
    .address(green_addr),
    .kernel11(green_kernel11),
    .kernel12(green_kernel12),
    .kernel13(green_kernel13),
    .kernel21(green_kernel21),
    .kernel22(green_kernel22),
    .kernel23(green_kernel23),
    .kernel31(green_kernel31),
    .kernel32(green_kernel32),
    .kernel33(green_kernel33),
    .pixel11(green_pixel11),
    .pixel12(green_pixel12),
    .pixel13(green_pixel13),
    .pixel21(green_pixel21),
    .pixel22(green_pixel22),
```

```
    .pixel23(green_pixel23),
    .pixel31(green_pixel31),
    .pixel32(green_pixel32),
    .pixel33(green_pixel33),
    .frame_sent(green_frame_sent)  // indicate one frame has been sent
);

conv_unit CONV_GREEN(
    .pixel_clk(clk25),
    .rst(rst),
    .enable(vga_data_en),
    .kernel11(green_kernel11),
    .kernel12(green_kernel12),
    .kernel13(green_kernel13),
    .kernel21(green_kernel21),
    .kernel22(green_kernel22),
    .kernel23(green_kernel23),
    .kernel31(green_kernel31),
    .kernel32(green_kernel32),
    .kernel33(green_kernel33),
    .pixel11( {1'b0, green_pixel11} ),
    .pixel12( {1'b0, green_pixel12} ),
    .pixel13( {1'b0, green_pixel13} ),
    .pixel21( {1'b0, green_pixel21} ),
    .pixel22( {1'b0, green_pixel22} ),
    .pixel23( {1'b0, green_pixel23} ),
    .pixel31( {1'b0, green_pixel31} ),
    .pixel32( {1'b0, green_pixel32} ),
    .pixel33( {1'b0, green_pixel33} ),
    .pixel_out(VGA_G)
);

//////////////////////////////////////////////////////////// \\
//////////////////// BLUE CHANNEL  ////////////////////////// \\
//////////////////////////////////////////////////////////// \\
wire [16:0] blue_addr;
wire [11:0] blue_data;

wire [3:0] blue_kernel11;
wire [3:0] blue_kernel12;
wire [3:0] blue_kernel13;
wire [3:0] blue_kernel21;
wire [3:0] blue_kernel22;
wire [3:0] blue_kernel23;
wire [3:0] blue_kernel31;
wire [3:0] blue_kernel32;
wire [3:0] blue_kernel33;

wire [3:0] blue_pixel11;
wire [3:0] blue_pixel12;
wire [3:0] blue_pixel13;
wire [3:0] blue_pixel21;
wire [3:0] blue_pixel22;
wire [3:0] blue_pixel23;
wire [3:0] blue_pixel31;
wire [3:0] blue_pixel32;
wire [3:0] blue_pixel33;

wire blue_frame_sent;

blk_mem_blue MEM_BLUE (
  .clka(clk25),        // input wire clka
  .ena(1'b1),          // input wire ena
  .wea(1'b0),          // input wire [0 : 0] wea
  .addra(blue_addr),   // input wire [16 : 0] addra
  .dina(),             // input wire [11 : 0] dina
  .douta(blue_data)    // output wire [11 : 0] douta
);

controller CNT_BLUE(
    .pixel_clk(clk25),
    .rst(rst),
    .data_en(vga_data_en),
    .data_in(blue_data),
    .address(blue_addr),
    .kernel11(blue_kernel11),
    .kernel12(blue_kernel12),
```

```
        .kernel13(blue_kernel13),
        .kernel21(blue_kernel21),
        .kernel22(blue_kernel22),
        .kernel23(blue_kernel23),
        .kernel31(blue_kernel31),
        .kernel32(blue_kernel32),
        .kernel33(blue_kernel33),
        .pixel11(blue_pixel11),
        .pixel12(blue_pixel12),
        .pixel13(blue_pixel13),
        .pixel21(blue_pixel21),
        .pixel22(blue_pixel22),
        .pixel23(blue_pixel23),
        .pixel31(blue_pixel31),
        .pixel32(blue_pixel32),
        .pixel33(blue_pixel33),
        .frame_sent(blue_frame_sent)  // indicate one frame has been sent
    );

    conv_unit CONV_BLUE(
        .pixel_clk(clk25),
        .rst(rst),
        .enable(vga_data_en),
        .kernel11(blue_kernel11),
        .kernel12(blue_kernel12),
        .kernel13(blue_kernel13),
        .kernel21(blue_kernel21),
        .kernel22(blue_kernel22),
        .kernel23(blue_kernel23),
        .kernel31(blue_kernel31),
        .kernel32(blue_kernel32),
        .kernel33(blue_kernel33),
        .pixel11( {1'b0, blue_pixel11} ),
        .pixel12( {1'b0, blue_pixel12} ),
        .pixel13( {1'b0, blue_pixel13} ),
        .pixel21( {1'b0, blue_pixel21} ),
        .pixel22( {1'b0, blue_pixel22} ),
        .pixel23( {1'b0, blue_pixel23} ),
        .pixel31( {1'b0, blue_pixel31} ),
        .pixel32( {1'b0, blue_pixel32} ),
        .pixel33( {1'b0, blue_pixel33} ),
        .pixel_out(VGA_B)
    );

endmodule
```

TESTBENCH CODE

```
`timescale 1ns / 1ps

module top_tb();

    reg  clk   = 0;
    reg  clk25 = 0;
    reg  rst   = 0;
    wire VGA_HS;
    wire VGA_VS;
    wire [3:0] VGA_R;
    wire [3:0] VGA_G;
    wire [3:0] VGA_B;

    top TOP_SYSTEM(
        .clk(clk),
        .rst(rst),
        .VGA_HS(VGA_HS),
        .VGA_VS(VGA_VS),
        .VGA_R(VGA_R),
        .VGA_G(VGA_G),
        .VGA_B(VGA_B)
    );

    always #5  clk   = ~clk;
    always #20 clk25 = ~clk25;
```

```
    parameter REFRESH = 416667;     // 1/60 second  refresh rate
    integer i   = 0;
    integer cnt = 0;

    integer fdr, fdg, fdb;

    initial
    begin
        fdr = $fopen("red.txt",   "w");
        fdg = $fopen("green.txt", "w");
        fdb = $fopen("blue.txt",  "w");

        repeat (200) @(posedge clk25);
        rst = 1;
        repeat (200) @(posedge clk25);
        rst = 0;

        // Write the output pixel values
        for( i = 0; i < REFRESH; i=i+1)
        begin
            @(posedge clk25);

            if(TOP_SYSTEM.vga_data_en)
            begin
                $fwrite(fdr, "%h", VGA_R);
                $fwrite(fdg, "%h", VGA_G);
                $fwrite(fdb, "%h", VGA_B);
                cnt = cnt + 1;
                if(cnt == 640)
                begin
                    $fwrite(fdr, "\n");
                    $fwrite(fdg, "\n");
                    $fwrite(fdb, "\n");
                    cnt = 0;
                end
            end
        end
        $fclose(fdr);
        $fclose(fdg);
        $fclose(fdb);
    end

endmodule
```

Utilization Report

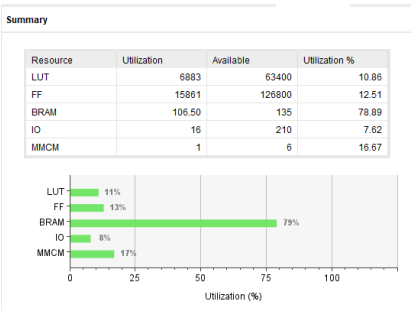

Figure 5.1: Utilization Report of Top Module

The design's placement on the target FPGA hardware and resource consumption were analyzed using the 'Utilization Report' obtained after the synthesis and implementation phases. Examination of the data (see relevant figure) reveals that the Look-Up Table usage, representing the system's logical processing load, is 10.86% (6883 units), and the Flip-Flop usage is 12.51% (15861 units). These low logic usage rates indicate that the controller and arithmetic units operate efficiently without straining the FPGA capacity. In contrast, Block RAM (BRAM)

usage is the most dominant resource in the system, at 78.89% (106.5 units). This high BRAM usage stems from the need for line buffering mechanisms and the storage of high-volume pixel data, inherent in image processing applications. In conclusion, the design efficiently utilizes the limitations of the FPGA's memory capacity while leaving ample room for future logical enhancements.

OUTPUT

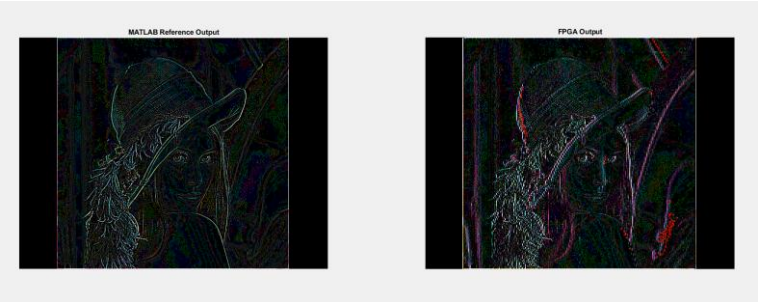

Figure 5.2: MATLAB Output of LENA

The FPGA design process was based on the MATLAB storage environment reference model for the purpose of comparing the accuracy of the hardware outputs. The MATLAB output presented on the left in the figure represents the ideal storage implementation of the 3x3 Laplacian filter. In this image, it can be seen that the high-frequency components of the temperature (where the edge is completely isolated) are suppressed, creating clear lines on a black background. This reference was used as a comparison standard to test the bit-level accuracy (bit-precise verification) of the FPGA design and to visually verify the Verilog encoding algorithm for separation.

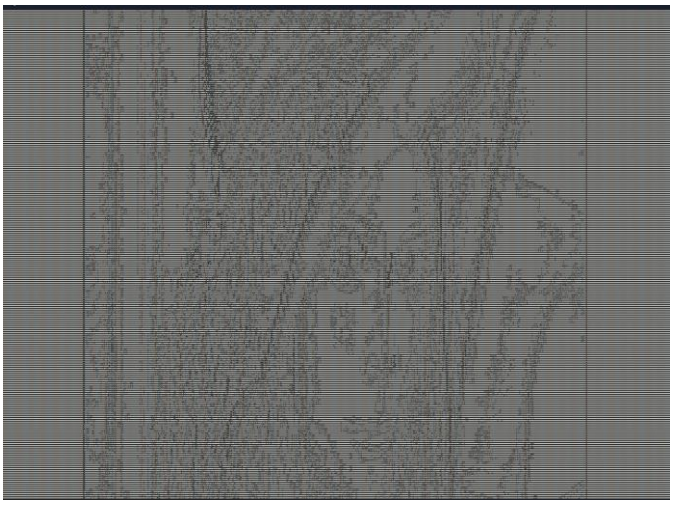

Figure 5.3: “red.txt” Output

In the final validation phase of the design, the pixel data processed by the FPGA convolution unit was saved by exporting it to a 'red.txt' file. When this dataset was visualized and compared

with the reference MATLAB outputs, it was found that the results almost perfectly matched the theoretical expectations. Examination of the output visualization shows that the sharp edges characteristic of the Laplacian filter were successfully created; object boundaries, hat curves, and facial details were as clear as in the reference image. After making corrections to data packaging and bit ordering, this clean result obtained from the red.txt file proves that the arithmetic operations and memory management on the hardware are working flawlessly and that the system fully meets the real-time image processing goals.